L2G2G: a Scalable Local-to-Global Network Embedding with Graph Autoencoders

Ruikang Ouyang[1], Andrew Elliott[3,4] Stratis Limnios[3], Mihai Cucuringu[2,3], and Gesine Reinert[2,3]

[1] Department of Engineering, University of Cambridge, UK
[2] Department of Statistics, University of Oxford, Oxford, UK
[3] The Alan Turing Institute, London, UK
[4] School of Mathematics and Statistics, University of Glasgow, Glasgow, UK

Abstract. For analysing real-world networks, graph representation learning is a popular tool. These methods, such as a graph autoencoder (GAE), typically rely on low-dimensional representations, also called *embeddings*, which are obtained through minimising a loss function; these embeddings are used with a decoder for downstream tasks such as node classification and edge prediction. While GAEs tend to be fairly accurate, they suffer from scalability issues. For improved speed, a Local2Global approach, which combines graph patch embeddings based on eigenvector synchronisation, was shown to be fast and achieve good accuracy. Here we propose L2G2G, a Local2Global method which improves GAE accuracy without sacrificing scalability. This improvement is achieved by dynamically synchronising the latent node representations, while training the GAEs. It also benefits from the decoder computing an only local patch loss. Hence, aligning the local embeddings in each epoch utilises more information from the graph than a single post-training alignment does, while maintaining scalability. We illustrate on synthetic benchmarks, as well as real-world examples, that L2G2G achieves higher accuracy than the standard Local2Global approach and scales efficiently on the larger data sets. We find that for large and dense networks, it even outperforms the slow, but assumed more accurate, GAEs.

Keywords: Graph Autoencoder, Local2Global, Node Embedding, Group Synchronisation

1 Introduction

Graph representation learning has been a core component in graph based real world applications, for an introduction see [13]. As graphs have become ubiquitous in a wide array of applications, low-dimensional representations are needed to tackle the curse of dimensionality inherited by the graph structure. In practice, low-dimensional node embeddings are used as efficient representations to address tasks such as graph clustering [25], node classification [2], and link prediction [27], or to protect private data in federated learning settings [20, 14].

Graph Autoencoders (GAEs) [23, 19] emerged as a powerful Graph Neural Network (GNN) [5] tool to produce such node representations. GAEs adapt autoencoders and variational autoencoders [1, 19] to graph structure data using a Graph Convolutional Neural Network (GCN) [28] as the encoder and for node embeddings. Although a GAE can achieve high accuracy in graph reconstruction, it suffers from a high computational cost. Several solutions for reducing computational workload have been proposed. Linear models, like PPRGo [4] and SGC [8], remove the non-linearities between layers. Layer-wise sampling methods such as GraphSage [12], FastGCN [6] and LADIES [29] sample a subset of neighbors of the nodes in each layer, while subgraph-sampling based methods such as GraphSaint [26] and Cluster-GCN [9] carry out message passing only through a sampled subgraph.

In this paper, we use FastGAE [22] as a starting point, which computes approximate reconstruction losses by evaluating their values only from suitably selected random subgraphs of the original graph. While FastGAE reduces the computational cost of a traditional GAE, its overall performance can be substantially inferior to a GAE when the sample used to approximate the loss is not large enough. For improved performance, the general Local2Global (L2G) framework by [15] leverages the eigenvector synchronization of [10, 11], to align independently created embeddings in order to produce a globally consistent structure (here we employ GAEs for embeddings and denote the resulting method by GAE+L2G). However, this architecture is data inefficient and suffers from a loss of performance in downstream tasks since it learns multiple separate GAEs. Moreover aggregating the local embeddings after the training process might lead to a loss of useful information learned during training.

Instead, we propose the Local to GAE to Global (L2G2G) model, which optimizes the Local2Global aligned embeddings directly, reducing the amount of information loss and allowing us to train a single global modified GAE. This structure leverages the scalable approach of FastGAE by only considering small sections of the graph when updating the weights. Figure 1 shows the model pipeline. Our main contributions are:
1. We introduce L2G2G as a new fast network embedding method.
2. We provide a theoretical complexity analysis for GAE+L2G and an experimental comparison of the runtimes showing that the runtime sacrifice for performance in L2G2G is minimal.
3. We test L2G2G and the baseline methods on real and synthetic data sets, demonstrating that L2G2G can boost the performance of GAE+L2G, especially on medium scale data sets, while achieving comparable training speed.

The paper is structured as follows. Section 2 introduces notation and discusses GAEs and the Local2Global framework by [15], including GAE+L2G. Section 3 presents our method, L2G2G, as well as a time complexity analysis, comparing it to GAE, FastGAE,and GAE+L2G. Section 4 provides experimental results on synthetic and real data sets, on networks of up to about $700,000$ nodes. In Section 5 we discuss the results and indicate directions for future work. The code is available at https://github.com/tonyauyeung/Local2GAE2Global.

sRx+T

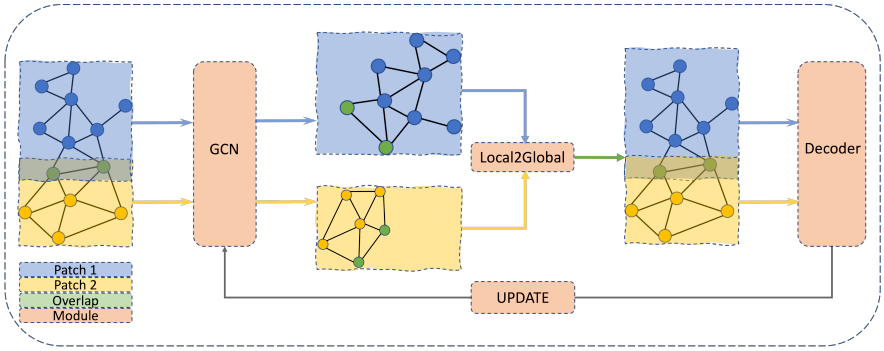

Fig. 1. L2G2L pipeline for two patches. The two patches are in blue and yellow, the overlapping nodes between them in green. Separate node embeddings for each patch are obtained via a single GCN. The decoder aligns the embeddings using the Local2Global synchronisation algorithm to yield a global embedding and then uses a standard sigmoid function. The GCN is then iteratively optimised using the training loss.

2 Preliminaries

Notations: An undirected attributed graph $G = (V, E, X)$ consists of a set of nodes V of size N, a set of unweighted, undirected edges E of size M, and a $N \times F$ matrix X of real-valued node attributes (features). The edge set is also represented by the $N \times N$ adjacency matrix A. Moreover, based on the L2G framework, we define a patch P to be a subgraph of G which is induced by a subset of the node set V; hence a patch P_i with the feature matrix corresponding to its nodes is denoted as $(V^{(i)}, E^{(i)}, X^{(i)})$ Node embeddings are denoted as a $N \times e$ matrix Z, where e is the embedding size and σ is the sigmoid function.

Graph Autoencoders (GAEs): Given a graph $G = (V, E, X)$, a GCN is used to obtain an $N \times e$ embedding matrix $Z = GCN(X, A)$, and a sparse approximation of the adjacency matrix through $\hat{A} = \sigma(ZZ^T)$. The GAE obtains the embedding through minimising the cross-entropy reconstruction loss, $L_{GAE}(\hat{A}, A) = Loss(\hat{A}, A) := -\sum_{i,j=1}^{N} A_{ij} log \hat{A}_{ij}$ with respect to the parameters of the GCN; this minimisation is also called *training*. Here a recursive method called *message passing* is used. The *decoder* then computes an inner product between each pair of node embeddings in the graph as proxy for the edge probabilities. Even though GAEs outperform traditional node embedding methods, such as spectral clustering [24] and DeepWalk [21], they usually scale poorly to large graphs. This is due to having to visit all the neighbors of a node recursively during the message passing phase in the encoding GCN, and the decoder scaling as $O(N^2)$ in complexity. We highlight two approaches for improving scalability:

1. FastGAE [22]: This model addresses the scalability issues by reconstructing the adjacency matrix of a sampled subgraph. This is achieved by evaluating an approximate reconstruction loss ($Loss_{approx}$) for every subgraph and aggregating them in one loss to be optimized by the model. This sampling procedure reduces

the computation complexity of decoding during each training epoch from $O(N^2)$ to $O(N_S^2)$, where N_S is the number of nodes in the subgraph.

2. Local2Global (L2G) [15]: This framework is a generic method to align embeddings computed on different parts of the graph (potentially on different machines and by different entities with different privacy constraints) into a single global embedding, regardless of the embedding method, as follows. Suppose that $P_1, ..., P_k$ are k patches, which pairwise overlap on at least d nodes and at most l nodes. It is assumed that the graph union of all patches gives the initial graph G. The pattern of overlapping patches is captured in a so-called *patch graph*, denoted $G_P = (V_P, E_P)$, whose node set $V_P = \{P_1, \ldots, P_k\}$ denote the patches. An edge between two nodes in G_P indicates that there is an overlap of at least d nodes in the initial graph G between those two patches. Then, for each patch P_i a node embedding matrix Z_i is computed using an embedding method of choice. When the embedding is obtained through a GAE, we refer to the method as GAE+L2G. Local2Global then leverages the overlap of the patches to compute an optimal alignment based on a set of affine transforms which synchronizes all the local patch embeddings into a single and globally consistent embedding, as follows. First, we estimate rotation matrices $\hat{S}_j \in \mathbb{R}^{F\times F}, j = 1, \ldots, k$, one for each patch. With $M_{ij} = \sum_{u\in P_i\cap P_j} X_u^{(i)}(X_u^{(i)})^T$ we first estimate the rotations between each pair of overlapping patches $(P_i, P_j) \in E_p$ by $R_{ij} = M_{ij}(M_{ij}^T M_{ij})^{-1/2}$. Next we build $\tilde{R}_{ij} = w_{ij}R_{ij}/\sum_j |V(P_i) \cap V(P_j)|$ to approximately solve the eigen problem $S = \tilde{R}S$, obtaining $\hat{S} = [\hat{S}_1, ...\hat{S}_k]$. We also find a translation matrix $\hat{T} = [\hat{T}_1, \ldots, \hat{T}_k]$ by solving $\hat{T} = \underset{T\in\mathbb{R}^{k\times F}}{\arg min}||BT - C||_2^2$, where $B \in \{-1, 1\}^{|E_p|\times k}$ is the incidence matrix of the patch graph with entries $B_{(P_i,P_j),t} = \delta_{it} - \delta_{jt}$, δ is the Kronecker Delta, and $C \in \mathbb{R}^{|E_p|\times F}$ has entries $C_{(P_i,P_j)} = \sum_{t\in P_i\cap P_j} \left(\hat{Z}_t^{(i)} - \hat{Z}_t^{(j)}\right)/|P_i \cap P_j|$. This solution yields the estimated coordinates of all the nodes up to a global rigid transformation. Next, we apply the appropriate rotation transform to each patch individually, $\hat{Z}^{(j)} = Z^{(j)}\hat{S}_j^T$, then apply the corresponding translation to each patch (hence performing translation synchronisation), and finally average in order to obtain the final aligned node embedding $\bar{Z}_i = \sum_j(\hat{Z}_i^{(j)} + \hat{T}_j)/|\{j \ : \ i \in P_j\}|$.

3 Methodology

Local-2-GAE-2-Global Combining Local2Global and GAEs produces a scalable GAE extension for node embeddings using autoencoders; using separate GAEs for each of the patches allows specialization to the unique structure in each of the patches. Our Local-2-GAE-2-Global (L2G2G) framework leverages the same divide-and-conquer technique Local2Global capitalises on, but is designed and adapted to the traditional GAE pipeline to benefit from its accuracy. The core idea of L2G2G is to evaluate embeddings locally on the patches but synchronizing the patch embeddings using the L2G framework while training a GAE. This leads to k GCNs encoding the k patches: $Z_i = GCN(X^{(i)}, A^{(i)})$, for $i = 1, \ldots, k$. To

account for the dynamic update during training and adapt to the local optimization scheme, we modify the GAE decoder to adjust the embeddings using the Local2Global framework; hence the patch-wise decoder in L2G2G estimates the edge probabilities between nodes in patches i and j by $\sigma((S_iZ_i+T_i)^T(S_jZ_j+T_j))$, where $S_i = S_i(Z)$ and $T_i = T_i(Z)$ are the Local2Global transformations of each of the patch embeddings.

In contrast to GAE+L2G, L2G2G synchronizes the embeddings before the decoder step and also performs synchronizations during the model training, thus taking full advantage of the patch graph structure during training. The cross-entropy losses of each patch are aggregated to give a global loss function:

$$L_{L2G2G} = \sum_{j=1}^{k} N_j L_{GAE}\left(\hat{A}^{(j)}, A^{(j)}\right)/N.$$

Similarly to the FastGAE algorithm, L2G2G reduces computation by only considering local structure. However, rather than training the network using only the local information, L2G2G aggregates the local embeddings to reconstruct the global information, thus boosting performance.

A schematic diagram of L2G2G is shown in figure 1, and pseudo-code for L2G2G is given in algorithm 1. As computing the alignment step can be costly, assuming that the Local2Global alignment would not change too quickly during training, we update the rotation and translation matrices only every 10 epochs.

Algorithm 1 Local-2-GAE-2-Global (L2G2G): An overview

Require: $P_1, ..., P_k$, where $P_j = (X^{(j)}, A^{(j)})$
  for e in $[1, ..., T]$ **do**
    for j in $[1, ...k]$ **do**
      $Z_j \leftarrow GCN(X^{(j)}, A^{(j)})$
    end for
    $\hat{Z}_1, ..., \hat{Z}_k \leftarrow Sync(Z_1, ..., Z_k)$
    $L \leftarrow 0$
    for j in $[1, ..., k]$ **do**
      $\hat{A}_j \leftarrow \sigma(\hat{Z}_j \hat{Z}_j^T)$
      $L \leftarrow L + \frac{N_j}{N} L_{GAE}(\hat{A}^{(j)}, A^{(j)})$
    end for
    Optimize encoder using L
  end for

Complexity Analysis Following the computations in [7] and [15], we derive the complexity of GAE, FastGAE, GAE+L2G and L2G2G. We assume that the number of nodes, edges and features satisfy $N, M, F \geq 1$, and, following [7], that the dimensions of the hidden layers in the GCN are all F. Then, the complexity of a L-layer GCN scales like $O(LNF^2 + LMF)$ and that of the inner product decoder scales like $O(N^2F)$. AE: maybe add something here about full decoder?

Thus, for as shown in [7], for T epochs the time complexity of the decoder and the encoder of a GAE scales like $O(T(LNF^2 + LMF + N^2F))$. In contrast, as stated in [22], the complexity of per-epoch of FastGAE with a $\sqrt{N}$ down-sampling size is $O(LNF^2 + LMF + NF)$, and hence for T epochs the FastGAE complexity scales like $O(T(LNF^2 + LMF + NF))$.

To simplify the complexity analysis of both Local2Global approaches we assume that the overlap size of two overlapping patches in the patch graph is fixed to $d \sim F$. Following [15], finding the rotation matrix S scales like $O(|E_p|dF^2) = O(|E_p|F^3)$. The translation problem can be solved by a t-iteration solver with a complexity per iteration of $O(|E_p|F)$, where t is fixed. To align the local embeddings, one has to perform matrix multiplications, which requires $O(N_jF^2)$ computations, where N_j is the number of nodes in the jth patch. The complexity of finding the rotation matrix ($O(|E_p|F^3)$) dominates the complexity of the computing the translation ($O(|E_p|F)$). Thus, the complexity of the L2G algorithm with k patches is $O\big(|E_p|F^3 + F^2 \sum_{j=1}^{k} N_j\big)$.

The GAE+L2G algorithm uses a GAE for every patch, and for the jth patch, for T training epochs the GAE scales like $O(T(LN_jF^2 + LM_jF + N_j^2F))$, with M_j number of edges in the jth patch. Summing over all patches and ignoring the overlap between patches as lower order term, so that $\sum_j N_j = O(N)$, $\sum_j N_j^2 \approx N^2/k$, and $\sum_j M_j \approx M$, the GAE+L2G algorithm scales like $O\left(TF\left(LNF + LM + N/k\right) + kF^3\right)$. For the complexity of L2G2G, as L2G2G aligns the local embeddings in each epoch rather than after training, we replace $kF^3 + NF^2$ with $T(kF^3 + NF^2)$, and thus the algorithm scales like $O\Big(T\Big(LNF^2 + LMF + \frac{N^2}{k}F + kF^3\Big)\Big)$. In the PyTorch implementation of FastGAE, the reconstruction error is approximated by creating the induced subgraph from sampling $\lfloor\sqrt{N}\rfloor$ proportional to degree, with an expected number of at least $O(M/N)$ edges between them. Then, the computation of the decoder is (at least) $O(M/N)$ instead of $O(N^2)$. Table 1 summarises the complexity results.

Model	General Time Complexity	PyTorch implementation
GAE	$O\left(TF(LNF + LM + N^2)\right)$	$O\left(TF(LNF + LM + M)\right)$
FastGAE	$\geq O\left(TF(LNF + LM + N)\right)$	$O\left(TF(LNF + LM + M/N)\right)$
GAE+L2G	$O\left(TF(L(NF + M) + \frac{N^2}{k}) + kF^3\right)$	$O\left(TF(LNF + LM + M) + kF^3\right)$
L2G2G	$O\left(TF(L(NF + M) + \frac{N^2}{k} + kF^3)\right)$	$O\left(TF(LNF + LM + M + kF^3)\right)$

Table 1. Complexity comparison in the general and in the sparse case .

Thus, in the standard case, increasing the number of patches k reduces the complexity of the computation of the GAE decoders. In the PyTorch implementation, if k scales linearly with N, the expression is linear in N. In contrast, when the number of nodes N is not very large, the number of features F becomes more prominent, so that the training speed may not necessarily increase with increasing number of patches. Table 1 shows that L2G2G sacrifices $O\left(TkF^3\right)$ training time to obtain better performance; with an increase in the number of patches, the training speed gap between L2G2G and GAE+L2G increases linearly.

4 Experimental Evaluation

Datasets To measure the performance of our method, we compare the ability of L2G2G to learn node embeddings for graph reconstruction against the following benchmark datasets Cora ML, Cora [3], Reddit [26] and Yelp [26].

In addition, we tested the performance of L2G2G on four synthetic data sets, generated using a Stochastic Block Model (SBM) which assigns nodes to blocks; edges are placed independently between nodes in a block with probability p_{in} and between blocks with probability p_{out} [17]. We encode the block membership as node features; with L blocks, v being in block l is encoded as unit vector $e_l \in \{0,1\}^L$. To test the performance across multiple scales we fix the number of blocks at 100, and vary the block size, p_{in} and p_{out}, as follows:

1. 'SBM-Small' with block sizes 10^2 and $(p_{in}, p_{out}) = (0.02, 10^{-4})$,
2. 'SBM-Large-Sparse' with block sizes 10^3 and $(p_{in}, p_{out}) = (10^{-3}, 10^{-4})$,
3. 'SBM-Large' with blocks of sizes 10^3 and $(p_{in}, p_{out}) = (0.02, 10^{-4})$,
4. 'SBM-Large-Dense' with block sizes 10^3 and $(p_{in}, p_{out}) = (0.1, 0.002)$.

Table 2 gives some summary statistics of these real and synthetic data sets.

	Stochastic block model				Real Data			
	Small	Large-Sparse	Large	Large-Dense	Cora ML	Cora	Reddit	Yelp
N	10,000	100,000	100,000	100,000	2,995	19,793	232,965	716,847
M	104,485	99,231	1,493,135	14,897,099	16,316	126,842	23,213,838	13,954,819
F	100	100	100	100	2,879	8,710	602	300

Table 2. Network data statistics: N = no. nodes, M = no. edges, F =no. features

Experimental setup and Results To assess whether L2G2G is a scalable alternative for the use of a GAE without having to sacrifice accuracy in downstream tasks, we compare it against the standard GAE [19], GAE+L2G [15] and Fast-GAE [22].We train the models on each data set for 200 epochs, with learning rate 0.001 and the Adam optimizer [18], and two layers in the GCN. The dimension of the first hidden layer is 32 and the dimension of the last layer is 16. We run each experiment 10 times with different random seeds for each model on each data set. All the experiments were conducted on a V100 GPU. We then compare the methods using the Area Under the Curve (AUC) and the Average Precision (AP). Following [15], we test our algorithm with fixed patch size 10.

Table 3 shows that L2G2G outperforms both FastGAE and GAE+L2G on most experiments. Having established the theoretical training speed gain of L2G2G, these results illustrate that L2G2G can perform better than the GAE+L2G, as well as achieve comparable training speed. AE: do we need to clarify with new pytorch stuff? Furthermore, in contrast to FastGAE we observe that the performance of L2G2G and of the GAE are very close to each other on the medium and large-scale data sets, indicating that L2G2G does not lose much performance compared to the much slower but assumed more accurate classic GAE. Furthermore, L2G2G even outperforms the GAE when the data set is large and dense, such as SBM-Large-dense and Reddit.

Average Performance On Different Datasets (AUC in %)				
	GAE	FastGAE	GAE+L2G	L2G2G
Cora ml	95.95 ± 0.42	83.90 ± 1.10	90.25 ± 0.19	**92.58 ± 0.35**
SBM-small	95.32 ± 0.18	76.34 ± 0.57	93.84 ± 0.14	**95.39 ± 0.21**
Cora	96.07 ± 0.09	81.78 ± 0.76	90.59 ± 0.11	**94.96 ± 0.26**
SBM-Large-sparse	94.88 ± 0.23	80.89 ± 0.84	94.73 ± 0.07	**95.02 ± 0.23**
SBM-Large	86.84 ± 0.11	70.90 ± 1.29	84.60 ± 0.10	**86.62 ± 0.25**
SBM-Large-dense	64.07 ± 0.12	65.20 ± 0.94	65.45 ± 0.05	**65.88 ± 0.03**
Reddit	88.69 ± 0.57	79.99 ± 1.31	**88.50 ± 0.23**	88.37 ± 0.39
Yelp	86.55 ± 0.28	73.79 ± 6.54	**85.82 ± 0.17**	84.01 ± 0.11

Average Performance On Different Datasets (AP in %)				
	GAE	FastGAE	GAE+L2G	L2G2G
Cora ml	95.37 ± 0.57	83.90 ± 1.10	90.57 ± 0.19	**92.41 ± 0.39**
SBM-small	95.23 ± 0.12	76.34 ± 0.57	95.22 ± 0.11	**95.71 ± 0.24**
Cora	95.76 ± 0.14	81.78 ± 0.76	90.50 ± 0.15	**94.67 ± 0.29**
SBM-Large-sparse	95.26 ± 0.24	80.89 ± 0.84	**95.88 ± 0.07**	95.44 ± 0.26
SBM-Large	89.64 ± 0.21	70.90 ± 1.29	87.35 ± 0.11	**89.34 ± 0.33**
SBM-Large-dense	67.64 ± 0.30	65.20 ± 0.94	71.25 ± 0.06	**72.08 ± 0.05**
Reddit	88.16 ± 0.60	79.99 ± 1.31	88.40 ± 0.18	**88.57 ± 0.40**
Yelp	86.73 ± 0.29	73.79 ± 6.54	**85.26 ± 0.12**	83.56 ± 0.11

Table 3. Experiments on different data sets with patch size 10. Bold: the best among the fast methods, underlined: the model outperforms the GAE.

Figure 2 shows a comparison of the training time of the models, as well as the changes of training speed as the data set size increases (on the log scale). It is worth mentioning that we are not accounting for the run time of the graph clustering. The results show that the training speed of L2G2G and GAE+L2G are very close on both small and large scale datasets. Although the gap between the training speed of L2G2G and that of GAE+L2G increases for large-scale data sets, L2G2G still achieves high training speed, and is even not much slower that FastGAE while achieving much better performance. In almost all cases, L2G2G is faster than the standard GAE, except for the two smaller datasets. Its training time is around an order of magnitude smaller per epoch for the larger models. As an aside, GAEs suffer from memory issues as they need to store very large matrices during the decoding step.

Ablation Study Here we vary the number of patches, ranging from 2 to 10. Figure 3 shows the performance changes with different number of patches for each model on each data set. When the patch size increases, the performance of L2G2G decreases less than GAE+L2G. This shows that updating the node embeddings dynamically during the training and keeping the local information with the agglomerating loss actually brings stability to L2G2G. Moreover, we have explored the behaviour of training time for L2G2G when patch size increases from 2 to 30, on both a small (Cora) and a large (Yelp) dataset. Figure 4

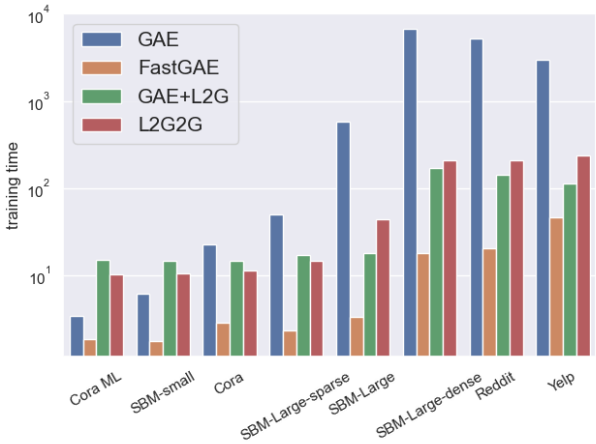

Fig. 2. Training time of the baseline models(GAE, FastGAE and GAE+L2G) and L2G2G on benchmark data sets (excluding partitioning time). Note that the y-axis is on a log-scale, and thus the faster methods are at least an order of magnitude faster.

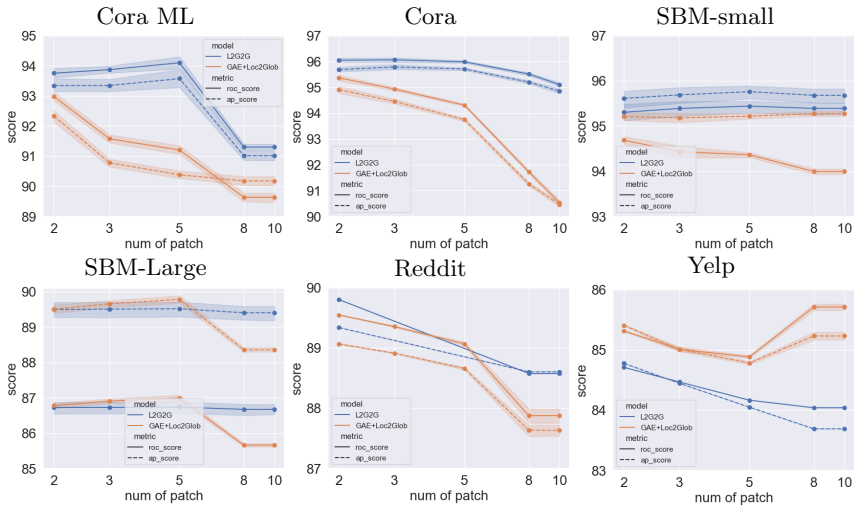

Fig. 3. Lineplots of the ROC score and accuracy of L2G2G and GAE+L2G, trained on each dataset, with different patch sizes. For each subplot, the blue lines represent the metrics for L2G2G, while the orange ones represent those for GAE+L2G. The shadows in each subplot indicate the standard deviations of each metric.

shows that on the small-scale data set Cora, the gap in training speed between L2G2G and GAE+L2G remains almost unchanged, while on Yelp, the gap between L2G2G and GAE+L2G becomes smaller. However, the construction of the overlapping patches in the Local2Global library can create patches that are

much larger than N/k, potentially resulting in a large number of nodes in each patch. Hence, the training time in our tests increases with the number of patches.

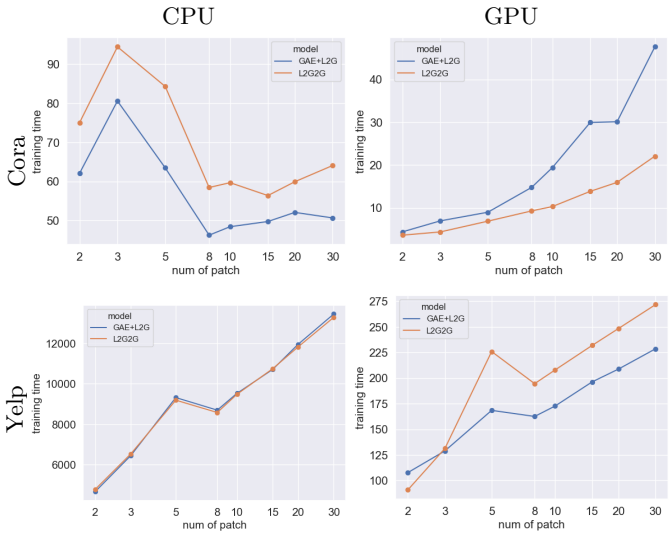

Fig. 4. Training time (excluding partitioning) of L2G2G and GAE+L2G on Cora (**Top**) and Yelp (**Bottom**), while varying patch size with CPU results presented on the **left** and GPU results presented on the **right**. The x axis is shown in log scale.

Since all the computations in Local2Global library built by [15] are carried out on the CPU, the GPU training can be slowed down by the memory swap between CPU and GPU. Thus, to further explore the behaviour of our algorithm when the number of patches increases, we ran the test on both CPU and GPU. The results are given by Figure 4. This plot illustrates that the GPU training time of L2G2G increases moderately with increasing patch size, mimicking the behaviour of GAE+L2G. In contrast, the CPU training time for the smaller data set (Cora) decreases with increasing patch size. The larger but much sparser Yelp data set may not lend itself naturally to a partition into overlapping patches. Summarising, L2G2G performs better than the baseline models across most settings, while sacrificing a tolerable amount of training speed.

5 Conclusion and Future Work

In this paper, we have introduced L2G2G, a fast yet accurate method for obtaining node embeddings for large-scale networks. In our experiments, L2G2G outperforms FastGAE and GAE+L2G, while the amount of training speed sacrificed is tolerable We also find that L2G2G is not as sensitive to patch size change as GAE+L2G.

Future work will investigate embedding the synchronization step in the network instead of performing the Local2Global algorithm to align the local embeddings. This change would potentially avoid matrix inversion, speeding up the calculations. We shall also investigate the performance on stochastic block models with more heterogeneity.

To improve accuracy, one could add a small number of between–patch losses into the L2G2G loss function, to account for edges which do not fall within a patch. The additional complexity of this change would be relatively limited when restricting the number of between–patches included. Additionally, the Local2Global library from [16] is implemented on CPU, losing speed due to moving memory between the CPU and the GPU. We will investigate re-implementing the Local2Global algorithm on a GPU.

References

1. Baldi, P.: Autoencoders, unsupervised learning, and deep architectures. In: I. Guyon, G. Dror, V. Lemaire, G. Taylor, D. Silver (eds.) Proceedings of ICML Workshop on Unsupervised and Transfer Learning, *Proceedings of Machine Learning Research*, vol. 27, pp. 37–49. PMLR, Bellevue, Washington, USA (2012)
2. Bayer, A., Chowdhury, A., Segarra, S.: Label propagation across graphs: Node classification using graph neural tangent kernels. In: ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 5483–5487. IEEE (2022)
3. Bojchevski, A., Günnemann, S.: Deep gaussian embedding of graphs: Unsupervised inductive learning via ranking. In: International Conference on Learning Representations (2018). URL https://openreview.net/forum?id=r1ZdKJ-0W
4. Bojchevski, A., Klicpera, J., Perozzi, B., Kapoor, A., Blais, M., Rózemberczki, B., Lukasik, M., Günnemann, S.: Scaling graph neural networks with approximate PageRank. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. ACM (2020)
5. Bruna, J., Zaremba, W., Szlam, A., LeCun, Y.: Spectral networks and locally connected networks on graphs. arXiv preprint arXiv:1312.6203 (2013)
6. Chen, J., Ma, T., Xiao, C.: Fastgcn: fast learning with graph convolutional networks via importance sampling. arXiv preprint arXiv:1801.10247 (2018)
7. Chen, M., Wei, Z., Ding, B., Li, Y., Yuan, Y., Du, X., Wen, J.: Scalable graph neural networks via bidirectional propagation. CoRR **abs/2010.15421** (2020). URL https://arxiv.org/abs/2010.15421
8. Chen, M., Wei, Z., Huang, Z., Ding, B., Li, Y.: Simple and deep graph convolutional networks. In: International Conference on Machine Learning, pp. 1725–1735. PMLR (2020)
9. Chiang, W.L., Liu, X., Si, S., Li, Y., Bengio, S., Hsieh, C.J.: Cluster-GCN. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. ACM (2019)
10. Cucuringu, M., Lipman, Y., Singer, A.: Sensor network localization by eigenvector synchronization over the euclidean group. ACM Trans. Sen. Netw. **8**(3) (2012)
11. Cucuringu, M., Singer, A., Cowburn, D.: Eigenvector synchronization, graph rigidity and the molecule problem. Information and Inference **1**(1), 21–67 (2012)

12. Hamilton, W., Ying, Z., Leskovec, J.: Inductive representation learning on large graphs. Advances in Neural Information Processing Systems **30** (2017)
13. Hamilton, W.L.: Graph representation learning. Morgan & Claypool Publishers (2020)
14. He, C., Balasubramanian, K., Ceyani, E., Rong, Y., Zhao, P., Huang, J., Annavaram, M., Avestimehr, S.: Fedgraphnn: A federated learning system and benchmark for graph neural networks. CoRR **abs/2104.07145** (2021). URL https://arxiv.org/abs/2104.07145
15. Jeub, L.G., Colavizza, G., Dong, X., Bazzi, M., Cucuringu, M.: Local2global: a distributed approach for scaling representation learning on graphs. Machine Learning **112**(5), 1663–1692 (2023)
16. Jeub, L.G.S.: Local2global github package. Github https://github.com/LJeub/Local2Global (2021)
17. Karrer, B., Newman, M.E.J.: Stochastic blockmodels and community structure in networks. Physical Review E **83**(1) (2011)
18. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: ICLR (Poster) (2015)
19. Kipf, T.N., Welling, M.: Variational graph auto-encoders. arXiv preprint arXiv:1611.07308 (2016)
20. Pan, Q., Zhu, Y.: Fedwalk: Communication efficient federated unsupervised node embedding with differential privacy. arXiv preprint arXiv:2205.15896 (2022)
21. Perozzi, B., Al-Rfou, R., Skiena, S.: DeepWalk. In: Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining. ACM (2014)
22. Salha, G., Hennequin, R., Remy, J.B., Moussallam, M., Vazirgiannis, M.: Fastgae: Scalable graph autoencoders with stochastic subgraph decoding. Neural Networks **142**, 1–19 (2021)
23. Simonovsky, M., Komodakis, N.: Graphvae: Towards generation of small graphs using variational autoencoders. In: Artificial Neural Networks and Machine Learning–ICANN 2018: 27th International Conference on Artificial Neural Networks, Rhodes, Greece, October 4-7, 2018, Proceedings, Part I 27, pp. 412–422. Springer (2018)
24. Tang, L., Liu, H.: Leveraging social media networks for classification. Data Min. Knowl. Discov. **23**(3), 447–478 (2011)
25. Tsitsulin, A., Palowitch, J., Perozzi, B., Müller, E.: Graph clustering with graph neural networks. arXiv preprint arXiv:2006.16904 (2020)
26. Zeng, H., Zhou, H., Srivastava, A., Kannan, R., Prasanna, V.: Graphsaint: Graph sampling based inductive learning method. In: International Conference on Learning Representations (2020)
27. Zhang, M., Chen, Y.: Link prediction based on graph neural networks. Advances in Neural Information Processing Systems **31** (2018)
28. Zhang, S., Tong, H., Xu, J., Maciejewski, R.: Graph convolutional networks: a comprehensive review. Computational Social Networks **6**(1), 1–23 (2019)
29. Zou, D., Hu, Z., Wang, Y., Jiang, S., Sun, Y., Gu, Q.: Layer-dependent importance sampling for training deep and large graph convolutional networks. Advances in neural information processing systems **32** (2019)